SpOdy

User manual — v0.1.0-alpha

Desktop frontend for the SpOdy orbital propagator.
Patran-style file-based workflow: fill a TOML, run the binary, inspect the output.

Table of contents

1. Introduction

SpOdy is a desktop application for propagating spacecraft and debris orbits around the Moon. You give it a mission scenario described in a plain-text file — central body, initial state, force model, integrator settings — and it produces a trajectory you can plot, compare against a reference, or sweep across thousands of variant cases in a batch run.

The application is shipped as a single self-contained folder. There is **no installer**, **no Python environment to configure**, and **no network registration** beyond the one-time download of the public ephemeris and gravity-model data the wizard handles for you on first launch. Extract the archive, run `spody-gui.exe`, and you are minutes away from your first propagation.

1.1 Who this manual is for

This manual assumes you have received a SpOdy bundle — a folder containing `spody-gui.exe` plus its supporting files — and want to understand how to use it. No prior orbital-mechanics expertise is required to follow the chapters in order, although a working knowledge of state vectors, Keplerian elements, and inertial frames will help you make the most of the analysis features.

If you are looking for the development guide (building from source, extending the integrator, contributing to the C core) you want a different document — that one ships separately.

1.2 What SpOdy does

At its core, SpOdy is a numerical orbit propagator with a desktop front-end. The split is deliberate: the integration physics lives in a fast C engine (`spody.exe`) that runs on its own as a command-line tool, and the graphical interface (`spody-gui.exe`) is a thin PySide6 application that orchestrates the inputs, dispatches the runs, and visualises the outputs. The two halves talk only through files — **the GUI never links C code directly**, which makes the same C engine usable from scripts, notebooks, or the upcoming batch automation features without any GUI surface area.

In one window you can:

- **Build** the simulation input through a structured form with live TOML preview, per-field range checks, and inline validation against the C parser, eliminating the need to remember any TOML syntax.
- **Run** a single propagation or a parameter sweep across thousands of cases, streaming the engine's terminal output into an embedded pane.
- **Analyse** the binary outputs through a catalogue of plots (state vectors, orbit shape, classical orbital elements, per-force acceleration breakdown, eclipse fractions, event timelines) in both 2D matplotlib and embedded 3D VTK canvases.
- **Compare** two runs side-by-side through dedicated difference plots, with automatic cubic-Hermite interpolation when the two output grids do not align sample-by-sample.

1.3 What is in the bundle

When you extract the distribution archive you get a folder structured like this:

```
spody-gui/
├── spody-gui.exe          ← the desktop application (this manual's subject)
├── spody.exe              ← the C engine spody-gui drives
├── data/                  ← populated by the setup wizard on first launch
├── examples/              ← starter TOML scenarios you can open
└── _internal/             ← bundled Python interpreter + libraries
```

The `_internal/` folder is an implementation detail of the PyInstaller bundling and you should treat it as opaque: never edit, delete, or rename files inside it. Everything you interact with is either the two executables, the example scenarios, or the wizard-populated `data/` folder.

1.4 How to read this manual

The first three chapters are sequential: chapter 2 walks you through the first launch and the setup wizard, chapter 3 introduces the main window and the run-time workflow. From chapter 4 onward the organisation switches to reference-style: schema details for every TOML field, a catalogue of every plot, frame conventions, and CLI flags for the `spody.exe` engine.

A consistent set of typographical conventions runs through the chapters:

- `monospaced text` denotes literal file paths, code snippets, and things you type or that the application displays verbatim;
- Ctrl + R shows a keyboard shortcut;
- **Bold** highlights UI controls (menu items, button labels, panel headings) as they appear in the application;
- *italics* introduce a new term the chapter will then use.

Throughout the manual you will also encounter call-outs in the margin:

> *Quotation blocks are used for verbatim excerpts of file content, log messages, or longer passages from the application's interface.*

Note, Tip, and Warning admonitions flag information that is, in order, useful background, a practical shortcut, and something you should not skip without thinking.

2. Installation and first launch

SpOdy is distributed as a single archive containing everything it needs to run. This chapter takes you from receiving that archive to seeing your first propagation results on screen, including the mandatory one-time setup wizard.

2.1 Prerequisites

The minimum requirements are unsurprising for a desktop scientific application:

- **Windows 10 (64-bit) or Windows 11**. The bundle is built for `x86_64`; ARM-based devices are not supported in this release.
- About **600 MB of free disk space** for the application itself, plus **roughly 30 MB to 350 MB** for the external data files the setup wizard downloads (the exact figure depends on which ephemeris coverage profile you choose — see chapter 3).
- A working **internet connection** the first time you run the application, so the setup wizard can fetch the planetary ephemeris and lunar gravity-model data from NASA's public servers. After the first run, SpOdy works fully offline.
- A graphics adapter that supports **OpenGL 3.2 or higher**. Any Windows-supported integrated GPU from the last decade is fine; the VTK-based 3D viewer falls back to software rendering when no hardware acceleration is available, with a small frame-rate cost.

You do **not** need Python installed: the bundle ships with its own Python interpreter plus every library it depends on. You also do not need administrator privileges to install or run SpOdy.

2.2 Installing

Installation in the conventional sense does not exist: there is no installer wizard, no Start menu entry created automatically, and no registry keys written. SpOdy is fully portable.

1. Locate the archive you received. It is named in the form `spody-gui-<version>-win64.zip` (or `.7z` for the smaller compressed variant).
2. Right-click the archive and pick **Extract All…** — or use any archiver such as 7-Zip if you prefer.
3. Choose a destination folder. SpOdy stores all its data inside the extracted folder, so pick somewhere you have write access:
   - **Your user folder**, for example `C:\Users\<you>\Apps\spody-gui\`, is the safest default.
   - **An external drive** (USB stick, network share) works too if you want a portable installation you can carry between machines.
   - `C:\Program Files\` is *not* recommended: the wizard needs to write the downloaded data files into the same folder, and Windows restricts write access there to administrators.
4. Open the extracted folder and confirm you can see at least these four items:

- `spody-gui.exe`
- `spody.exe`
- `data/` (empty at this stage)
- `_internal/` (do not touch)

The bundle is now ready to launch.

2.2.1 Verifying the bundle (optional)

If you want to make sure the archive transferred without corruption before running anything, the distribution comes with a `SHA256SUMS` text file listing the expected SHA-256 hash of each top-level file. From a PowerShell prompt opened in the extraction folder:

```
Get-FileHash spody-gui.exe -Algorithm SHA256
Get-FileHash spody.exe     -Algorithm SHA256
```

Compare the hexadecimal values against the matching lines in `SHA256SUMS`. Mismatches mean the download was truncated or tampered with — redownload from the original source.

2.3 First launch

Double-click `spody-gui.exe`. After a brief warm-up the main window appears, with two tabs at the top (**Run** and **Analysis**) and an empty menu bar.

Because no data files are present yet, you will immediately see two modal pop-ups:

1. An informational dialog announcing **"Setup needed"** with the path to the data folder it expects to populate (always `data/` relative to `spody-gui.exe`).
2. Following that, the **Setup wizard** itself.

The wizard is the topic of the next chapter. For now it is enough to know that until you complete it, the application will refuse to launch any propagation: a *hard run guard* checks that all required data files are present before every run and will pop the wizard back up if any are missing.

> *The first launch can take 5 to 10 seconds before the window becomes responsive while the bundled Python interpreter and the Qt framework warm up. Subsequent launches are typically under a second — the operating system caches the bundle in memory.*

2.4 Settings file location

SpOdy persists a handful of user preferences (the path to `spody.exe`, the data-folder override, the recent-files list, the 3D Moon texture choice, the last-selected ephemeris coverage profile) through the standard Windows registry under `HKEY_CURRENT_USER\Software\SpOdy\SpOdy`.

Nothing inside the bundle folder is modified by writing settings, so the bundle remains fully portable: you can copy or move the folder to a different machine, and the application will start fresh with the wizard on first launch there, ignoring whatever settings remain in the old machine's registry.

To **reset SpOdy to a clean state** without uninstalling, delete the registry key above (via `regedit`) and the `data/` folder next to the executable. The next launch will behave exactly as if you had just extracted the bundle.

2.5 Uninstalling

Drag the extracted folder to the Recycle Bin. That is the entire uninstallation procedure. There is nothing else to clean up except the registry key mentioned above, which is harmless if left behind.

3. The setup wizard

A fresh SpOdy bundle cannot run a propagation. Two large data files — a planetary ephemeris and a lunar gravity-model — are needed before the first integration step, and neither is shipped inside the archive (they are externally maintained, publicly distributed, and together comprise hundreds of megabytes that change rarely). The setup wizard is the dialog that downloads those files into the bundle's `data/` folder, then converts the raw ephemeris into the internal binary format the engine expects. You run it once at first launch and never think about it again, unless you choose later to widen the ephemeris coverage or upgrade to a different gravity model.

3.1 When the wizard appears

The wizard pops in three situations:

1. **At first launch**, when no data files are present in the bundle's `data/` folder.
2. **At any later launch**, when one or more required files are missing or corrupt (this can happen if you delete the `data/` folder by hand, or if the previous download was interrupted and left a half-written file).
3. **On demand**, when you choose **Settings › Setup wizard…** from the menu bar. You typically reach for this when you want to switch ephemeris coverage profiles, add an extra gravity model, or simply re-verify that everything is in order.

While the wizard is open, the rest of the application keeps running in the background. You can close the wizard at any time with the **Close** button at the bottom-right; nothing you have downloaded so far is lost.

3.2 What the wizard manages

The wizard is structured as a list of *assets*, one card per file it needs to put on disk. Each card shows:

- a **status icon** indicating whether the file is present (`✓` green), absent (`✗` red), or present but too small to be trustworthy (`⚠` amber);
- the asset's **human name** and, if present, its on-disk size;
- the **download URL** in an editable field (more on this in section 3.4);
- a **progress bar** (mostly idle until you press Download);
- a **Download** button on the right that toggles to **Cancel** while a transfer is in flight.

The assets fall into two categories:

Raw assets are files SpOdy fetches verbatim from public servers:

- the JPL `DE440` planetary ephemeris (header + one or more ASCII chunks); the wizard offers two coverage profiles, see section 3.2;

- the GRGM1200B lunar harmonic-gravity coefficients (the `.tab` file with the spherical-harmonic coefficients and a small `.lbl` metadata sidecar from the PDS Geosciences Node).

Derived assets are files SpOdy produces from raw ones on your machine:

- `de440.spody`, the internal binary format the engine expects. Built once from the downloaded DE440 ASCII chunks by invoking the `spody.exe convert ephemeris` subcommand under the hood. The conversion runs **automatically** as soon as the raw inputs are complete; you never click a button for it.

3.3 Choosing an ephemeris coverage profile

The DE440 planetary ephemeris is split into ~100-year chunks. SpOdy gives you two pre-set profiles, picked through a radio at the top of the wizard:

Profile	Chunks	Coverage window	Download size
Modern era *(default)*	1 chunk (`ascp01950.440`)	1950 – 2050	~30 MB
Full pack	11 chunks (`ascp01550..ascp02550`)	1550 – 2650	~340 MB

The default is *Modern era*. It is the right pick for anyone running near-present scenarios, which means almost everyone reading this manual: any simulation epoch from 1950 to 2050 is covered exactly, with the same accuracy as the full pack. The full pack only matters for historical reconstructions (lunar landings of the late 1960s, the Apollo program, deep-time analyses of orbital secular drift) or far-future scenarios.

Switching coverage at any time is non-destructive: changing the radio rebuilds the asset list to reflect the new requirement, but nothing is deleted from disk. So if you start with *Modern era*, later upgrade to *Full pack*, then change your mind, the extra chunks remain on disk and the wizard simply stops requiring them. The derived `de440.spody` always includes every chunk it finds in the `DE440/` folder, regardless of profile, so you never lose coverage by toggling.

> *Storage for the Full pack profile is dominated by the eleven ASCII chunks (~30 MB each); the derived `de440.spody` from the full set is about 100 MB on disk.*

3.4 Downloading the data

Press **Download** on a single card to fetch that file. For a fresh install the fast path is the **Download all missing** button in the wizard's footer: it walks every card whose state is `✗` or `⚠` and starts the download. Cards in flight show their progress in the bar; you can cancel one without affecting the others by clicking the **Cancel** button that replaces **Download** during a transfer.

Failed downloads do not leave corrupted files behind: SpOdy writes each transfer to a `.part` sidecar first and renames it to the final name only after the HTTP response completes successfully. If the connection drops mid-transfer, the next download attempt starts over rather than appending to a stale partial.

3.4.1 Why the URL field is editable

The URLs SpOdy ships with point at the canonical sources we have verified work at the time of the release:

- JPL's FTP-over-HTTPS server for the DE440 ASCII chunks;
- the PDS Geosciences Node at Washington University for the GRGM1200B `.tab` and `.lbl` files.

Both servers are stable, public, and well-maintained, but URLs do move over time. The **editable URL field** on each card lets you paste a different location if a download fails — for instance, if NASA mirrors the file under a different path after a server reorganisation, or if your organisation hosts an internal mirror behind a firewall. The wizard does not persist URL overrides: they apply only to the current dialog session. Once you confirm a new URL works, please share it with the SpOdy maintainers so the next release ships with the corrected default.

3.5 Conversion to the internal format

Once the wizard sees that every required *raw* DE440 file is present and the derived `de440.spody` is either missing or older than the most recent raw chunk, it triggers the conversion automatically. You will see the **Conversion (auto)** status line at the bottom of the wizard switch through three states in sequence:

1. *waiting on raw DE440 (…)* — while downloads are still in flight;
2. *converting… 01950* — the engine is reading the ASCII chunks (the chunk IDs are listed live);
3. *de440.spody ready (98.4 MB)* — success.

The conversion itself takes a few seconds for the *Modern era* profile and a few tens of seconds for the *Full pack*. It uses the `spody.exe convert ephemeris` subcommand under the hood; see chapter 12 if you want to run the same conversion manually from a shell.

3.6 When everything is green

A complete bundle has every card showing a green ✓ and a status line reading *de440.spody ready* with a size in megabytes. At that point you can close the wizard with the **Close** button and start using the application: opening one of the example TOML files, or building a new scenario from scratch in the Run tab.

3.7 Re-opening the wizard later

The wizard remains available at any time through **Settings › Setup wizard…** in the menu bar. The two most common reasons to reopen it later are:

- **Widening coverage**: you started with *Modern era* but now want to run an Apollo-era scenario, so you flip to *Full pack* and click **Download all missing** to fetch the additional historical chunks.
- **Re-verifying after a copy**: you moved the bundle to a new machine and want a quick visual confirmation that nothing got lost along the way. The wizard's status icons answer that instantly.

You can also open the wizard purely to **change the data folder** through the **Change…** button at the top. SpOdy persists the override in the registry and uses it from then on; the bundle's `data/` folder is no longer touched. This is useful if you keep multiple SpOdy bundles around but want them to share a single copy of the (potentially hundreds of megabytes of) downloaded data.

4. The main window

Once the wizard has done its job you can take stock of the application proper. This chapter is a tour of the main window: the two top-level modes (**Run** and **Analysis**), the menus, the status bar, and how the pieces work together over the course of a typical session. Detailed coverage of the form, the schema, the plot catalogue, and the diff workflow waits for the chapters that follow.

4.1 Layout overview

The window divides into four regions:

1. The **menu bar** along the top, with five menus: **File**, **Edit**, **Run**, **Settings**, **Help**.
2. The **mode tab strip** immediately below it, with two tabs: **Run** and **Analysis**. The window switches between completely different layouts depending on which tab is active; the menus stay shared, although a few entries are no-ops while the irrelevant tab is up.
3. The **active workspace**, which fills the bulk of the window and changes shape with the mode (see sections 4.2 and 4.3).
4. The **status bar** along the bottom, with the path of the currently loaded TOML on the left and the run state (`idle`, `running 12.3s`, `OK (47.1s)`, `exit 1 (3.2s)`) on the right.

The window remembers its last size and position across launches through Qt's standard mechanism, persisted alongside the other settings in the registry.

4.2 The Run tab

The Run tab is where you describe a scenario, validate it, and launch the engine. Its workspace is a horizontal splitter you can drag to redistribute width:

Pane (left)	Pane (right)
TOML form (chapter 5)	Terminal view streaming spody's output
Live TOML preview below	Engine status (idle / running)

On the left, an automatically-generated form lets you fill in every field the TOML schema accepts — one widget per scalar key, collapsible sections for optional blocks (`[spacecraft.srp]`, `[events]`, `[batch]`), and a live TOML preview pane underneath showing exactly what will be written to disk when you press **Generate**.

On the right, a read-only terminal view displays the merged stdout and stderr streams of `spody.exe` while it runs. Output appears line by line as the engine produces it, exactly as you would see it in a console; no ANSI colour codes are interpreted, because the engine does not produce any.

At the very top of the form sits a row of action buttons:

- **Load…**: open a `.toml` file and pre-fill the form with its values.
- **Generate**: serialise the current form state to disk as canonical TOML. The first time, you are prompted for a filename; subsequent presses overwrite the same file.
- **Validate**: write a temporary TOML next to the current one and invoke `spody.exe validate` synchronously, displaying the result as a coloured badge under the row (`✓ valid` in green, or `✗ <reason>` in red with the full error in the tooltip).
- **RUN** (green): generate, then launch `spody.exe propagate` or `spody.exe batch` (the form auto-detects which subcommand applies from the presence of a `[batch]` section).

Detailed coverage of the form and the schema appears in chapters 5 and 6.

4.3 The Analysis tab

The Analysis tab is where you inspect simulation outputs. Its workspace is a horizontal splitter with a richer left column:

Pane (left, vertical splitter)	Pane (right)
File tree (auto-scanned)	Sun-arrow row (only when 3D plot)
Add external button	Stacked 2D / 3D canvas
Overlay selected button	Info label
(splitter)	
Plot tree (grouped by topic)	
Tile selected button	

The left column is split vertically: file selection on top, plot selection at the bottom, with a draggable bar between them. The right column is dedicated to the canvas, with the optional Sun-arrow row appearing only when a 3D plot is active.

The **working directory** at the top of the tab is the folder the file tree scans for `.bin` outputs (recursively, up to three levels deep). It auto-fills when you load a TOML in the Run tab, pointing at the TOML's parent so the analysis results land naturally next to their input. You can also change it manually via the **Change…** button.

Click a single file in the file tree to load it. The application detects its type from the 8-byte magic in the header (`SPDYOUT_` trajectory, `SPDYACC_` accelerations, `SPDYEVT_` events) and rebuilds the plot tree with the catalogue applicable to that type. Click a leaf in the plot tree to render it immediately into the canvas. Re-click the same leaf to re-plot.

Detailed coverage of the analysis layout, including overlay and diff workflows, appears in chapters 8 through 11.

4.4 The menus

The menu bar is identical in both modes. Entries that do not apply to the current mode are still listed but are no-ops until you switch.

4.4.1 File

- **New** — reset the Run-tab form to a blank scenario.
- **Open…** — open a `.toml` file (also achievable via the form's **Load…** button).
- **Open Recent ›** — jump to one of the last eight files you have opened. **Clear list** at the bottom of the submenu empties the history.
- **Save** / **Save As…** — write the current form state to the current path, or to a chosen path.
- **Quit**.

4.4.2 Run

- **Validate**, **Propagate**, **Batch** — the same three engine subcommands available through the buttons on the form, exposed through keyboard shortcuts (Ctrl+T, Ctrl+R, Ctrl+B respectively).
- **Stop** (Ctrl+.) — terminate the current engine run. The signal sequence is graceful first, hard kill after two seconds.

4.4.3 Settings

- **Paths…** — the persistent paths dialog: where to find `spody.exe`, the data dir, the Moon texture used by the 3D viewer.
- **Setup wizard…** — reopen the wizard (chapter 3) at any time, even when everything is already green.
- **About** — version and build info for both halves of the application.

4.4.4 Help

- **User manual** — opens this document.
- **Visit project page** — opens the SpOdy project landing page in your default browser, if a network is available.

4.5 The status bar

The status bar at the very bottom of the window shows two permanently visible items:

- on the **left**, the absolute path of the TOML the Run tab is currently editing (or the literal string `(unsaved)` if you have never saved). An asterisk after the path (`<path>*`) indicates unsaved edits in the form;

- on the **right**, the engine's current state:
- `idle` — no run in progress;
- `running 14s` — a run is in flight; the elapsed-time counter updates every second;
- `OK (47.1s)` (green) or `exit 1 (3.2s)` (red) — the result of the last completed run.

The status bar is purely informational; nothing in it is clickable.

4.6 A typical session

To put the layout in motion: a routine session of SpOdy looks like this.

1. Launch `spody-gui.exe`. The window opens on the Run tab with an empty form (or with whichever file you last saved, if Open Recent is non-empty and you click one).
2. Either build a scenario from scratch by typing into the form, or open one with **File › Open…** or the **Load…** button at the top of the form.
3. Click **Validate** to check the schema. The badge under the button turns green; if it does not, fix the indicated field and try again.
4. Click **RUN**. The Run tab's right pane streams the engine's output for the duration of the propagation; the status bar ticks `running 12s`, `running 13s`, …
5. When the run finishes, the status bar shows `OK` in green. You are now ready to inspect the results: switch to the **Analysis** tab. Its working directory has auto-pointed at your TOML's folder, so the freshly produced `.bin` files are already listed in the file tree.
6. Click a `.bin` file to load it; click a plot leaf to render it; use **Ctrl/Shift-click** for multi-selection if you want to overlay or tile several plots.

That is the loop. Everything else — batches, diffs, 3D scenes, frame conventions — is variations on it.

5. Building a simulation: the TOML form

The Run tab's left pane is a *form* over the TOML schema: one widget per scalar key, collapsible groups for optional sections, range checks at every keystroke, and a live preview of the canonical TOML the form will write to disk. This chapter walks through how to fill it in. The schema details — types, ranges, defaults, what each field means physically — live in chapter 6.

5.1 Anatomy of the form

The form is a vertical scroll of section groups, one per top-level TOML section. From top to bottom in the order spody expects them:

1. `[simulation]`
2. **Object** — either `[spacecraft]` *or* `[debris]` (mutually exclusive; the choice is made through a pair of radio buttons at the top of the group)
3. `[initial_state]`
4. `[force_model]`
5. `[ephemeris]`
6. `[integrator]`
7. `[output]`
8. `[events]` *(optional, enabled by checkbox)*
9. `[batch]` *(optional, enabled by checkbox)*

Each section header is the literal TOML name, so the mapping between the form and the file you produce is one to one. The form never invents fields and never silently drops fields it does not understand: anything it loads from a TOML file that it does not have a widget for is preserved verbatim through a *passthrough* mechanism and re-emitted on the next save.

5.2 Filling a field

Every field type follows the same pattern: a label on the left, a widget on the right.

- **Strings** (`name`, `frame`, `central_body`, …): a plain line edit, except for known enumerated values that use a combo box (e.g. `frame` accepts only `central_inertial` today).
- **Floats** (`mass_kg`, `et_start_s`, `rel_tol`, …): a line edit, no validator attached (so the text you type stays verbatim — no surprise normalisation of `1.0e-5` into `1e-05`). Range checks happen at every keystroke against the field's registered validator (see section 5.3).
- **Integers** (`harmonics_degree`, `thread_number`): a line edit with a `QIntValidator` enforcing the min/max range as you type.
- **Booleans** (`srp`): a checkbox.
- **Vector-of-three** (`position_km`, `velocity_kms`): three line edits side by side.

- **Lists of strings** (`third_bodies`): one checkbox per known value (`Sun`, `Mercury`, `Venus`, `Earth`, …).
- **Paths** (`harmonics_file`, `ephemeris.file`, `output.csv_file`, …): a line edit alongside a **Browse…** button that pops a file dialog and writes the chosen path back into the edit.

Hovering the cursor over a label or its widget shows a **tooltip** with the field's one-line description; range-validated fields also include the allowed range in the tooltip.

5.3 Validation as you type

A field whose value falls outside the registered range is flagged **immediately** with a red border and a small warning glyph appended to its tooltip (`⚠ must be > 0`, `⚠ must be in [2, 1200]`). The form does not block invalid input — you can continue editing — but the visual cue makes the bad field obvious.

The range checks the form runs are local: they catch values that are physically impossible (negative masses, zero rel-tol) or violate a documented bound (harmonics degree above 1200). Cross- field consistency (e.g. `h_init_s` between `h_min_s` and `h_max_s`) is *not* checked by the form — that is the engine's job, and **Validate** is the right button to press once the per-field indicators look clean.

5.4 The XOR object switch

Two TOML sections describe the propagated object, and a propagation must use exactly one of them. The form models this with a pair of radio buttons at the top of the **Object** group:

- **Spacecraft** — the conventional case: a body with a known mass and (when SRP is enabled) a cross-sectional area. This is the `[spacecraft]` section in the emitted TOML.
- **Debris** — an inferred body where only the area-to-mass ratio matters, because the gravity-driven accelerations are mass- independent and SRP acceleration depends only on `A/m`. This is the `[debris]` section.

Switching the radio shows the matching widgets and hides the others. The form never emits both sections; whichever radio is up at **Generate** time is the one that ends up in the file.

The choice also affects which override targets are available in the `[batch.columns]` mapping table (chapter 7). For instance, `spacecraft.srp.area_m2` only appears as a target when the radio is on Spacecraft, and `debris.am_srp` only when it is on Debris.

5.5 Optional sub-blocks

Three sections are not always required, and the form gates them behind a checkbox:

- `[spacecraft.srp]` — the SRP cannonball sub-block of `[spacecraft]`. Enable the *Enable [spacecraft.srp]* checkbox to expose the sub-form; inside it a second pair of radios chooses between `area_m2` (with `A/m = area / mass`) and `am_srp` (the ratio specified directly).

- `[events]` — opt-in eclipse detection through `eclipse_threshold`. The threshold field is gated behind the *Enable [events]* checkbox.
- `[batch]` — the multi-case sweep block. Enable the *Enable [batch]* checkbox to expose the batch-specific form (name, output directory, thread count, cases CSV, column mapping table). Batch scenarios are covered in detail in chapter 7.

Disabling any of these checkboxes after filling in fields *hides* the widgets but does not erase the values: re-enable later and your typed numbers are still there. At emit time, however, a disabled block is omitted entirely — the form behaves as if you had never filled it. This is the safest behaviour for round-tripping files that contain sections you do not currently want to touch.

5.6 Conditional UI

A few smaller conditionals are wired into the form to keep the visible surface relevant:

- `output.interval_s` is only shown when `output.mode = "fixed"`. In step mode it is meaningless (the integrator decides when to emit a record), so the row hides itself and the field is stripped from the emitted TOML even if you typed a value previously.
- The `spacecraft.srp.area_m2` and `spacecraft.srp.am_srp` fields grey each other out depending on which SRP-parameter radio is selected. The parser rejects files that set both, so this prevents the most common XOR mistake before it happens.

5.7 Save versus Generate

Two paths produce the same canonical TOML on disk:

- the **File › Save** (and **Save As…**) menu items;
- the **Generate** button at the top of the form.

The difference is purely conceptual: **Save** reads as “I’m done editing”, **Generate** reads as “give me the TOML so I can do something with it” (typically the **RUN** button immediately following). Functionally they go through the same emitter, produce byte-identical output, and update the same recent-files list.

The emitter is **schema-aware**: it knows the canonical order of sections and keys, formats floats with `repr()` precision, and emits inline tables for the entries inside `[batch.columns]` when they use delta mode. Two **Generate** clicks on the same form state produce byte-identical files, so the result diffs cleanly between runs.

5.8 The live TOML preview

Below the form (the lower half of the splitter visible in the Run tab’s left pane) lives a read-only TOML preview. It mirrors what **Generate** would write to disk, refreshed on every keystroke.

Practically, it gives you three things:

1. a sanity check that the widgets you have just touched produce the TOML you expect — especially useful when learning the schema or when you want to see how the form serialises an optional sub-block;
2. a copy-friendly view of the canonical output, so you can paste the result into another tool, a chat message, or a bug report without having to round-trip through disk;
3. a feedback loop for the **passthrough** mechanism: any section the form does not directly render but found in the loaded file appears at the bottom of the preview, confirming it is being carried through to the next save.

If you ever type something the emitter cannot serialise (very rare — only `ValueError` from an internal conversion would trigger it) the preview shows a single-line `# (form has invalid values: …)` placeholder until the next valid edit.

5.9 Validating with the engine

The **Validate** button at the top of the form runs the engine's own parser against the form's current state, *without writing your real file*. Internally the form:

1. produces the canonical TOML from `to_dict()` (the same path the live preview uses);
2. writes it to a temporary file next to your current saved file (or to the OS temp dir if you have not saved yet) so that relative paths inside the TOML (`harmonics_file`, `ephemeris .file`, `batch.cases_file`) resolve identically to what they would at run time;
3. calls `spody.exe validate <tempfile>` and waits up to 30 seconds for the result;
4. deletes the temp file and displays the verdict as a coloured badge to the right of the button:
   - `✓ valid` in green when the engine returns exit code 0;
   - `✗ <last line of stderr>` in red otherwise, with the full message in the tooltip.

Any edit you make after a green badge clears it back to neutral, so a stale green never misleads you into thinking an out-of-sync file passed validation.

6. TOML schema reference

This chapter is the field-by-field reference for the TOML input files SpOdy reads. Every section, every key, every accepted value is listed here in the order they appear in the form. For the high-level workflow consult chapter 5; for batch-specific keys also see chapter 7.

The conventions used in the tables below:

- **Type** is the TOML type the engine expects, with units where applicable.
- **Default** is what the engine assumes when the key is absent. An em dash (—) marks keys that are required.
- **Range** documents the allowed values; an unbounded numeric field is shown as `> 0` or similar.

6.1 `[simulation]`

Scenario name and time window. Required.

Key	Type	Default	Range	Description
`name`	string	—	–	Human-readable scenario name. Used as the prefix for batch case output names.
`et_start_s`	float	—	–	Start epoch as TDB seconds past the J2000 epoch (2000-01-01 12:00:00 TT). Negative values are valid for pre-J2000 epochs.
`duration_s`	float	—	`> 0`	Propagation duration in seconds. Positive only (forward-time propagation).

The `et_start_s` value is the same scale the planetary ephemeris uses internally. Converting a calendar date to ET seconds past J2000 is the user's responsibility; the engine does not perform any UTC/TAI/TDB conversions. The DE440 wizard data covers 1950 – 2050 by default; choose the *Full pack* coverage profile in the wizard if you need to start outside that window.

6.2 `[spacecraft]` *or* `[debris]`

Mutually exclusive object descriptions. Exactly one of the two sections must be present in a valid TOML.

6.2.1 `[spacecraft]`

The conventional case: a body with a known dry mass. Gravity is mass-independent, but SRP scales as `A/m`, so when SRP is enabled the engine derives `A/m` from the area and the mass.

Key	Type	Default	Range	Description
`mass_kg`	float	—	`> 0`	Dry mass in kilograms.

The optional `[spacecraft.srp]` sub-table is detailed below.

6.2.2 `[debris]`

The inferred-body case: only the area-to-mass ratio matters. Use this section when you do not have or do not care about a mass value, typically for parameter sweeps over a debris cloud.

Key	Type	Default	Range	Description
`am_srp`	float	—	`> 0`	Area-to-mass ratio in m²/kg, used by SRP.
`Cr`	float	`1.5`	`>= 0`	Reflectivity coefficient, only consulted when SRP is enabled. `1.0` = pure absorbing, `2.0` = pure mirror.

In Debris mode, every batch override target that mentions a mass or area (`spacecraft.mass_kg`, `spacecraft.srp.area_m2`) is unavailable; only `debris.am_srp` and `debris.Cr` are accepted.

6.2.3 `[spacecraft.srp]` (optional)

The cannonball solar-radiation-pressure sub-block. Present only when `[spacecraft]` is the active object and the *Enable [spacecraft.srp]* checkbox is ticked.

Within this sub-block exactly one of `area_m2` and `am_srp` is allowed; setting both is a validation error.

Key	Type	Default	Range	Description
`area_m2`	float	—	`> 0`	Cross-sectional area in m². The engine derives `A/m = area_m2 / mass_kg`.
`am_srp`	float	—	`> 0`	Area-to-mass ratio in m²/kg, specified directly. Equivalent to `area_m2 / mass_kg`; use this form when you want sweep over `A/m` independently of `mass_kg`.
`Cr`	float	`1.5`	`>= 0`	Reflectivity coefficient, same convention as in `[debris]`.

6.3 `[initial_state]`

Initial position and velocity vector of the propagated object. Required.

Key	Type	Default	Range	Description
`frame`	string	—	`central_inertial`	Reference frame. Only `central_inertial` is supported in this release (the central body's J2000-aligned inertial frame; see chapter 10).
`position_km`	array of 3 floats	—	–	`[x, y, z]` position in km in the chosen frame.
`velocity_kms`	array of 3 floats	—	–	`[vx, vy, vz]` velocity in km/s, same frame as `position_km`.

The initial state must be self-consistent: an `|r|` smaller than the central body's mean radius will trigger an IMPACT event at the first step. A `|v|` greater than the local escape velocity turns the simulation into a hyperbolic flyby, which the engine handles but is rarely what the user intended; double-check the magnitudes against your scenario.

6.4 `[force_model]`

Forces the propagator integrates against. Required.

Key	Type	Default	Range	Description
`central_body`	string	—	`Moon`	Central body of the propagation. Only `Moon` is supported in this release.
`harmonics_file`	string (path)	—	–	Path to a spherical-harmonic gravity coefficients file (`gggrx_1200b_sha.tab` for the recommended GRGM1200B model). Relative paths resolve against the TOML's directory.
`harmonics_degree`	int	—	`[2, 1200]`	Truncation degree of the harmonic gravity expansion. Higher = more accurate but more expensive. See *Choosing a harmonics degree* below for guidance.
`third_bodies`	array of strings	`[]`	one of `Sun`, `Mercury`, `Venus`, `Earth`, `Moon`, `Mars`, `Jupiter`, `Saturn`, `Uranus`, `Neptune` (excluding the central body)	Perturbing bodies whose point-mass gravity is added at every step.
`srp`	bool	`false`	–	Enable cannonball SRP. When `true` a `[spacecraft.srp]` block must be present (in Spacecraft mode) or `am_srp` must be set in `[debris]` (in Debris mode).

6.4.1 Choosing a harmonics degree

A rough guide for runs with the Moon as the central body, based on empirical scaling of the GRGM1200B model and the cost of the spherical-harmonic evaluation (which scales as $O(N^2)$):

N	Use case
30 – 50	quick sanity propagation, low-fidelity orbit averaging
80	reasonable default for LRO-class missions; sub-km vs SPICE LRO POD over 6 days
150	sweet spot for low-lunar orbits; recovers ~95% of N=200's residual reduction at half the cost
200	high-fidelity floor; beyond ~200 the GRGM1200B coefficients become weakly observed and adding terms can slightly *increase* mean drift

Higher N values are accepted (up to the model's nominal 1200) but do not visibly improve accuracy for the example scenarios shipped with SpOdy.

6.5 `[ephemeris]`

Path to the planetary ephemeris binary. Required.

Key	Type	Default	Range	Description
`file`	string (path)	—	–	Path to a `.spody` ephemeris file. Use `de440.spody` produced by the setup wizard (chapter 3). Relative paths resolve against the TOML's directory.

A future release may accept `.bsp` SPICE kernels directly; today only the internal `.spody` format is supported.

6.6 `[integrator]`

Integration algorithm and tolerances. Required.

Key	Type	Default	Range	Description
`type`	string	—	`rkdp45`	Integration scheme. Only Dormand-Prince 5(4) is supported in this release.
`rel_tol`	float	—	`> 0`	Relative error tolerance per accepted step. `1e-11` is the recommended default for orbital regression work.
`h_init_s`	float	—	`> 0`	Initial step size in seconds. Normally somewhere between `h_min_s` and `h_max_s`.
`h_min_s`	float	—	`> 0`	Minimum allowed step size. The integrator gives up and reports failure if it would need to step smaller than this.
`h_max_s`	float	—	`> h_min_s`	Maximum allowed step size. Useful as a guard against the integrator picking very large steps in low-perturbation regions and missing events.

A typical low-lunar-orbit setup uses `rel_tol = 1e-11`, `h_init_s = 60`, `h_min_s = 1e-5`, `h_max_s = 2700`. The relatively large `h_max_s` (45 minutes) is harmless because the adaptive controller picks smaller steps where the dynamics need them.

6.7 `[output]`

Output stream configuration: which files to write, and at what cadence. Required.

Key	Type	Default	Range	Description
`mode`	string	—	`fixed` or `step`	Output cadence. `fixed` writes records on a uniform grid (`interval_s`); `step` writes one record per accepted RKDP step.
`interval_s`	float	—	`> 0`	Sampling interval in seconds when `mode = "fixed"`. Ignored otherwise.
`csv_file`	string (path)	none	–	CSV trajectory output. Empty/absent = no CSV produced.
`bin_file`	string (path)	none	–	Binary (`SPDYOUT_`) trajectory output. Recommended for analysis: the GUI's reader path uses this format.
`log_file`	string (path)	none	–	Path that the engine tees its stdout/stderr into.
`accelerations_file`	string (path)	none	–	Per-force acceleration breakdown (`SPDYACC_` format). Empty = no breakdown produced.
`events_log`	string (path)	none	–	Event records (`SPDYEVT_` format) for impacts and (if `[events]` is enabled) eclipses.

All output paths are resolved relative to the TOML's directory, so a TOML at `examples/foo/input.toml` with `bin_file = "output/foo.bin"` writes the binary into `examples/foo/output/foo.bin`. You are responsible for creating the destination directory; the engine does not create it for you.

6.8 `[events]` (optional)

Opt-in event detection. Present only when the *Enable [events]* checkbox is ticked.

Key	Type	Default	Range	Description
`eclipse_threshold`	float	—	`[0, 1]`	Sunlight-fraction crossing that fires an eclipse event. `0` = enter umbra (start of total eclipse); `1` = full sunlight (end of any eclipse); `0.5` = penumbra midpoint.

Impact events are always detected regardless of this section: any trajectory that crosses a central-body or third-body surface (mean radius) produces an IMPACT record. The `[events]` section only controls the opt-in eclipse detection.

6.9 `[batch]` (optional)

Multi-case sweep section. Present only when the *Enable [batch]* checkbox is ticked. Covered in detail in chapter 7.

Key	Type	Default	Range	Description
`name`	string	—	–	Batch run name. Used as the prefix for per-case output names.
`output_dir`	string (path)	—	–	Existing directory under which a `batch/` subfolder is auto-created.
`thread_number`	int	`1`	`[1, N_cpu]`	Worker thread count. `1` = sequential. Capped at the host's logical-CPU count by the form. Parallel execution requires the OpenMP-enabled engine build.
`cases_file`	string (path)	—	–	CSV file describing the parameter sweep. First non-comment line is the header.
`columns`	inline table	empty	–	Mapping from CSV columns to target paths (see chapter 7). Programmatic; the form's column-mapping table is the friendly editor.

7. Batch propagations

A *batch* is a parameter sweep: the same scenario propagated many times with one or more fields varied case by case. SpOdy describes the sweep through a CSV file (one row per case, columns matching the fields to vary) and a mapping table that tells the engine which CSV column overrides which TOML field. This chapter walks through how to set one up.

7.1 Why batches

The typical reasons to run a batch instead of repeated single-case runs are:

- **Sensitivity analyses** — sweep over `harmonics_degree` to see how much accuracy you actually need; sweep `Cr` to understand the SRP coefficient's contribution to long-term drift; sweep `mass_kg` to characterise the response of a chosen propellant tank load.
- **Monte-Carlo simulation** — randomly perturb the initial state across thousands of cases and compute statistics on the final position.
- **Debris-cloud propagation** — one case per fragment, each with its own `A/m` ratio drawn from a debris-population distribution.

The engine threads cases across CPU cores when the OpenMP-enabled build is used and `batch.thread_number > 1` (see section 7.5), giving near-linear speed-up for the typical case sizes.

7.2 The CSV file

A SpOdy batch cases file is a plain CSV. The conventions are:

- Lines starting with `#` are comments and are skipped.
- The **first non-comment, non-blank line is the header**: a comma-separated list of column names. Whitespace around each name is trimmed.
- One special header value is reserved: `id`. When present, the column's value is used as the per-case name (for the output file basenames). When absent, the engine auto-generates names like `case_0001`, `case_0002`, …
- Every other column is a **parameter override** whose name is arbitrary; the mapping to a TOML field is decided by the `[batch.columns]` section, not by the column name itself.
- All non-`id` cells are parsed as floats.

An example three-case CSV for a mass + Cr sweep:

```
id, mass_kg, Cr
A,  1916.0,  1.3
B,  1916.0,  1.5
C,  2500.0,  1.3
```

Three cases (`A`, `B`, `C`) with two parameters varied.

7.3 The column mapping table

The form's `[batch]` section embeds a table that maps every non-`id` CSV column to a *target*: a dotted path inside the TOML schema. When the engine runs case *i*, it reads row *i* of the CSV, applies each cell to the target the column is mapped to, and propagates the resulting scenario.

The table updates live whenever you change the **cases_file** field above it:

- you can browse to a CSV with the **Browse…** button, which also re-reads the column list immediately;
- or type the path manually and press **Re-read columns**;
- or load a TOML that already has a `[batch.columns]` block: the table populates from the file.

For each row of the table you see three cells:

CSV column	Target dropdown	Mode dropdown

The **Target** dropdown contains all valid override paths for the currently-selected object mode (Spacecraft or Debris), pre-filled with a heuristic when the column name matches the last segment of a known target (`mass_kg` ⇒ `spacecraft.mass_kg`, `am_srp` ⇒ `debris.am_srp`). The **Mode** dropdown picks between the two override semantics described in section 7.4 below. A column you leave on `(unassigned)` is silently dropped at **Generate** time — the engine validates that the table covers every non-`id` column at run time, so unassigned columns trigger a clean error.

7.3.1 Available targets

The list of override paths is the same as the path-style API the engine accepts for `[batch.columns]`:

- `simulation.et_start_s`, `simulation.duration_s`
- `spacecraft.mass_kg`, `spacecraft.srp.area_m2`, `spacecraft.srp.Cr` (only in Spacecraft mode)
- `debris.am_srp`, `debris.Cr` (only in Debris mode)
- `initial_state.position_km[0]`, `[1]`, `[2]`
- `initial_state.velocity_kms[0]`, `[1]`, `[2]`
- `force_model.srp`
- `integrator.rel_tol`, `integrator.h_init_s`, `integrator.h_min_s`, `integrator.h_max_s`
- `output.interval_s`

Component access into the vector-of-three fields (`position_km[0]`) lets you sweep along a single axis without disturbing the others.

7.4 Override vs delta

The **Mode** dropdown picks between two semantics for the CSV value:

- `override` (default) — the value in the CSV cell *replaces* the field's TOML value. If your TOML has `mass_kg = 1916.0` and a row's `mass_kg` column reads `2500.0`, the case runs with `mass_kg = 2500.0`.
- `delta` — the cell value is *added* to the field's TOML value. The same row with `mass_kg` in delta mode would run with `mass_kg = 1916.0 + 2500.0 = 4416.0`. Useful for Monte-Carlo perturbations around a nominal case, where the CSV column is literally a delta drawn from a distribution.

In delta mode the emitted TOML uses an *inline table* for the column descriptor:

```
[batch.columns]
mass_kg = "spacecraft.mass_kg"                          # override
mass_dm = { target = "spacecraft.mass_kg", mode = "delta" }
```

Mixing both modes across columns is supported. Mixing across *cases* (i.e. the same column behaving as delta in one row and override in the next) is not.

7.5 The data preview

Below the mapping table, a small read-only table previews the first ten data rows of the cases CSV verbatim, with the original column names (including `id`) as headers. It is purely informational, but two things make it worth glancing at:

- you can sanity-check that the column-to-target mapping is right by comparing each header against the value you see — `mass_kg` values around 1916 should belong to a mass column;
- on long CSVs the row counter `(preview: first 10 of N rows)` next to the preview tells you how many total cases the engine will see, useful for catching truncated files.

7.6 Threading

The `batch.thread_number` field caps automatically at the host's logical-CPU count (the form's integer validator enforces this as you type, with the cap shown next to the field). Setting more threads than cores never helps and often hurts (oversubscription), so the form refuses values above the cap rather than letting them through silently.

A few additional points:

- **Sequential execution** (`thread_number = 1`) works with any engine build. If you do not know whether your `spody.exe` was built with OpenMP, leave the field at `1` and the batch will still run correctly, just one case at a time.

- **Parallel execution** (`thread_number > 1`) requires an OpenMP- enabled engine. The standard SpOdy bundle ships with one such build. The engine rejects the run with a clean message if the build does not support parallel execution.
- **Determinism**: the engine processes cases in a fixed order regardless of thread count, so the per-case output is bit- identical across thread counts.

7.7 Output layout

When the batch runs, the engine creates a `batch/` subfolder inside `batch.output_dir` and writes one set of files per case inside it:

```
<output_dir>/batch/
  <name>_<case_id>.csv
  <name>_<case_id>.bin
  <name>_<case_id>_acc.bin   (if accelerations_file is set)
  <name>_<case_id>_evt.bin   (if events_log is set)
```

The `<name>` part comes from `batch.name`; the `<case_id>` part is the value of the `id` column or the auto-generated `case_NNNN` name. Existing files in `batch/` are overwritten without warning — the engine assumes you have moved or archived any previous runs you wanted to keep.

7.8 A complete worked example

The `examples/batch_demo/` scenario shipped with SpOdy is a realistic, runnable batch. Open `examples/batch_demo/input.toml` in the Run tab to see the form populated:

- a single `[simulation]` block;
- `[spacecraft]` with a nominal `mass_kg = 1916.0`;
- `[spacecraft.srp]` enabled with a nominal `Cr = 1.3` and `area_m2 = 15.0`;
- `[batch]` enabled with `cases_file = "cases.csv"`, `name = "mass_srp_sweep"`, and a column mapping that overrides `spacecraft.mass_kg` and `spacecraft.srp.Cr`.

The CSV at `examples/batch_demo/cases.csv` lists three cases (`A`, `B`, `C`) varying mass and reflectivity, exactly as in section 7.1. Press **RUN** to dispatch all three; the Run tab's terminal pane streams the case-by-case progress, and the resulting files land in `examples/batch_demo/output/batch/`. Switch to the Analysis tab afterwards: the working directory has auto-pointed at `examples/batch_demo/output/`, all three trajectories appear in the file tree, and you can `Ctrl`-click them all and press **Overlay selected** on a single-line plot (such as **Radial distance |r|**) to compare the cases visually.

8. The analysis tab

The Analysis tab is where you inspect the binary outputs the engine produces. Its workspace is organised around two trees on the left (files and plots) and a canvas on the right that switches between a 2D matplotlib pane and a 3D VTK pane depending on what you ask to render. This chapter walks through the layout and the interaction patterns; the plot catalogue itself lives in chapter 9.

8.1 Layout

The Analysis tab consists of four regions:

1. The **working directory** row at the top, with a path field, a **Change…** button, and a **Refresh** button that rescans the folder.
2. The **left column**, a vertical splitter with two halves:
   - **upper half**: the file tree, with `+ Add external file...` and `→ Overlay selected` buttons at the bottom;
   - **lower half**: the plot tree, with the `▦ Tile selected (N)` button at the bottom. The splitter bar between the two halves is draggable; pull it up to give the plot tree more room when many plots are visible, pull it down when you want more file rows.
3. The **right column**, a vertical stack:
   - a **Sun-arrow row** at the top that appears only when the active plot is 3D (chapter 9 covers what it does);
   - the **canvas** itself (matplotlib for 2D plots, VTK for 3D);
   - an **info label** at the bottom, with the current file or operation summary.
4. The **horizontal splitter** between left and right columns is draggable too.

The window-size and splitter positions are not yet persisted across launches in this release; default sizes apply on every launch.

8.2 The working directory

Almost everything in the Analysis tab is anchored to a *working directory* — a folder the application scans recursively (up to three levels deep) looking for `.bin` files. The file tree's **In folder** section is auto-populated from this scan.

The working directory updates **automatically** in two situations:

- when you load a TOML in the Run tab (it points at the TOML's parent folder, so the spody output ends up listed right next to its input);
- when a run completes (the Refresh-after-run mechanism re-scans the folder so new files appear immediately).

You can also set it manually with **Change…**, or refresh the current scan with the ⟳ **Refresh** button after, for example, producing files outside of SpOdy.

8.3 The file tree

The file tree has two top-level sections:

1. **In folder (`<working_dir>`)** — everything the scan found, listed by path relative to the working directory.
2. **External (N)** — explicit files you added via **+ Add external file…**. These remain in the list across working-directory changes, so you can keep a reference file visible regardless of which run-output folder you are looking at.

Each leaf shows the basename of a `.bin` file. Hovering a leaf shows the absolute path. Adding an *external* file via the button also loads it immediately (single-click load), which is a small convenience — you typically add an external file because you want to look at it next.

8.3.1 Selection modes

The file tree supports two interaction modes:

- **Single click** on a leaf — loads that file. The application detects its type from the 8-byte magic in the header and rebuilds the plot tree with the catalogue applicable to that file's kind (`SPDYOUT_` trajectory, `SPDYACC_` accelerations, `SPDYEVT_` events).
- **Ctrl-click / Shift-click** — extends the selection (Qt's *ExtendedSelection* mode). The currently-loaded file (the one whose data feeds the active plot) does not change. Multi- selection is the input for the **Overlay** and **Diff** buttons.

8.4 The plot tree

The plot tree below the file splitter is the catalogue of plots applicable to the currently-loaded file's kind. It is grouped by topic (for trajectories: *State vectors*, *Orbit shape*, *Orbital elements*, *Diff (pick 2 files)*) with collapsible folders. Single- click a leaf to render it into the canvas; re-clicking the same leaf re-plots.

Three things to know about it:

1. **Click is dispatch** — there is no "Plot" button to press after picking a leaf. The plot fires on the click.
2. **Ctrl-click extends** — the *Tile selected* button below the tree uses the multi-selection (chapter 9, section 9.4).
3. **The Sun-arrow row appears only for 3D plots** — the row immediately above the canvas hides itself when the active plot is 2D, because the arrow has no meaning there. When a 3D plot is selected the row reappears with its epoch field and **+ Sun arrow** button.

The plot tree is rebuilt every time you load a different *kind* of file (e.g. switching from a trajectory to an accelerations breakdown). Within the same kind it persists across loads.

8.5 Overlaying multiple files

Selecting multiple files in the file tree (Ctrl/Shift-click) and pressing → **Overlay selected** plots them together on a single axes, with a turbo-coloured palette and a legend listing each file basename.

The overlay only applies to plots that draw **one line per file**. A plot that already draws three lines (the per-component `Position x, y, z`, `Velocity vx, vy, vz`, the per-force accelerations breakdown) is not overlay-safe; selecting it and pressing **Overlay** triggers an explanation dialog instead of an illegible 3N-line plot.

The button works for the **3D orbit + Moon** plot too. In that case the overlay paints each trajectory as its own polyline in the same VTK scene, with a viewport legend and **Ctrl+click picking** enabled on every polyline (more on picking in chapter 9, section 9.7).

A few rules:

- All selected files must be the **same kind** as the currently loaded one. Files of a different kind are quietly skipped and listed under *Skipped* in the info label.
- The overlay tolerates **different time grids** as long as the underlying plot uses each file's own t-axis (e.g. `|r|(t)` plots each file's own `t` independently). Time-aligned overlay is not available; for sample-aligned comparison use the diff plots.
- The **Overlay** button silently ignores **Diff (pick 2 files)** specs — clicking a diff plot is its own dispatch pathway and does not need an overlay button.

8.6 Tiling several plots

The ▦ **Tile selected (N)** button below the plot tree is the dashboard mode. Multi-select N plot leaves with Ctrl/Shift-click (the counter on the button updates live), then click. The canvas splits into `ceil(sqrt(N)) × ceil(N/cols)` subplots, one per selected plot, all rendered against the currently-loaded file.

Two modes are auto-detected from the selection:

- **All single-file plots** — the subplots draw against the loaded file. Useful for at-a-glance overview: select |r|, |v|, a, e, i, Ω (six leaves) and the result is a 2×3 dashboard of the most relevant orbit quantities.
- **All diff plots** — the subplots draw against the two files selected in the *file tree* (with the same selection rule as a single diff click). Read disks once, render four diff subplots showing |Δr|, |Δv|, components, and RIC decomposition.

Mixed sets (some single, some diff) are refused with a clear message. **3D plots** are filtered out and the count of skipped 3D plots is reported in the info label. The hard cap is **12 subplots** to keep the result legible; selecting more triggers a warning and aborts the tile.

8.7 Picking in the 3D viewer

When a 3D plot is active, **Ctrl+left-click** on any rendered trajectory polyline picks it: the line is highlighted, the matching file is selected (highlighted) in the file tree on the left, and the info label below the canvas reports the picked file's path. Pointing at the central body (the Moon sphere) does nothing. This is mostly useful in overlay scenes, where ten lines may cross each other and you want to know which file produced which.

The pick interaction does not change which file is loaded, only which one is *highlighted*. Click a file in the tree (or Ctrl- click anywhere on a polyline) to keep working with it.

9. Plot catalogue

This chapter is the reference for every plot SpOdy ships with. The entries are grouped first by file kind (trajectory, accelerations, events) and within each by the topic folder the plot tree shows them under. Each entry documents what the plot displays, the formulas used, whether the plot is *overlay-safe* (a single line per file, so an N-file overlay produces N lines rather than 3N), and whether it is single- or two-file (*diff*).

Notational conventions throughout the chapter:

- `t` — time elapsed from the start of the propagation, in seconds.
- `r, v` — position and velocity in km and km/s, in the central-body inertial frame (chapter 10).
- `|x|` — Euclidean norm of vector `x`.
- The horizontal axis of any *(t)* plot is the trajectory's own time column, in seconds. The toolbar at the top of the canvas lets you zoom, pan, and save the current view as PNG.

9.1 Trajectory plots (`SPDYOUT_`)

9.1.1 State vectors

Radial distance |r|

Distance of the spacecraft from the central body's centre, as a function of time.

Formula. `|r|(t) = sqrt(x^2 + y^2 + z^2)`.

When to use. Quickest sanity check that the orbit is in the right altitude band. For LRO, the average altitude is about 50 km above the Moon's mean radius of 1737.4 km, so `|r|` oscillates around 1788 km.

Overlay-safe. Yes.

Speed |v|

Magnitude of the inertial velocity vector.

Formula. `|v|(t) = sqrt(vx^2 + vy^2 + vz^2)`.

When to use. Companion to `|r|(t)` — for an elliptical orbit, `|v|` oscillates inversely to `|r|` per the vis-viva relation.

Overlay-safe. Yes.

Position x, y, z

The three Cartesian components of the position vector, on a shared axes.

When to use. When you want to see which component is the dominant driver of an altitude excursion, or to spot a slow secular drift on a single axis (typical of certain harmonics-driven perturbations).

Overlay-safe. No (draws three lines per file).

Velocity vx, vy, vz

Same as the position components but for the velocity vector.

Overlay-safe. No.

9.1.2 Orbit shape

XY projection (and XZ, YZ)

Projection of the orbit onto the named coordinate plane of the inertial frame, with a green dot at `t = 0` and a red dot at the final sample.

When to use. Visualises the orbit's geometric shape and the sense of motion (the trajectory line goes from green to red along the orbit's direction). For a near-polar lunar orbit the XY plot looks like a tight loop; the XZ and YZ projections show the inclination directly.

Overlay-safe. Yes.

3D orbit + Moon

The trajectory rendered as a polyline in a 3D scene with the central body sphere at the origin. The sphere is textured with the equirectangular image configured in **Settings › Paths › Moon texture** (or rendered grey if no texture is configured).

Interactions.

- Left-drag rotates the camera.
- Scroll zooms in and out.
- Middle-drag pans the camera.
- `r` resets the camera to the default oblique view.
- `Ctrl`+left-click on a polyline picks it (in overlay mode).

The Sun-arrow row above the canvas applies to this plot. Type an epoch (TDB seconds past J2000) into the field and click **+ Sun arrow** to add an arrow pointing from the central body toward the Sun at that epoch. The epoch field auto-fills with the currently-loaded TOML's `simulation.et_start_s` so it usually contains a sensible value already.

Overlay-safe. Yes (the overlay variant adds N trajectories in turbo colours plus a legend).

9.1.3 Orbital elements

The six entries in this folder are the classical Keplerian elements computed from the state vectors at every sample. The shared implementation handles the two degenerate cases gracefully: in a circular orbit (`e ≈ 0`) the argument of periapsis and the true anomaly are set to 0; in an equatorial orbit (`i ≈ 0`) the RAAN is set to 0 and its rotation is folded into the argument of periapsis. The thresholds are tight enough (`1e-8`) that any realistic propagated orbit is unaffected.

The default central-body gravitational parameter is the Moon's (`mu = 4902.800066 km^3/s^2`).

Semi-major axis a

Formula. From the vis-viva relation: `1/a = 2/|r| - |v|^2 / mu`.

Units. km.

Overlay-safe. Yes.

Eccentricity e

Formula. Magnitude of the Laplace-Runge-Lenz vector: `e_vec = (v × h) / mu - r/|r|`, where `h = r × v` is the specific angular momentum.

Units. Dimensionless.

Overlay-safe. Yes.

Inclination i

Formula. `cos(i) = h_z / |h|`.

Units. Degrees.

Overlay-safe. Yes.

RAAN Ω

Formula. With `n = z_hat × h = (-h_y, h_x, 0)` the node line, `cos(Ω) = n_x / |n|`, quadrant resolved by the sign of `n_y`.

Units. Degrees, range `[0, 360)`.

Overlay-safe. Yes.

Argument of periapsis ω

Formula. `cos(ω) = (n · e_vec) / (|n| |e_vec|)`, quadrant resolved by the sign of `e_vec_z`.

Units. Degrees, range `[0, 360)`.

Overlay-safe. Yes.

True anomaly ν

Formula. `cos(v) = (e_vec · r) / (|e_vec| |r|)`, quadrant resolved by the sign of `r · v`.

Units. Degrees, range `[0, 360)`.

Note. For a multi-revolution propagation `v(t)` shows the saw-tooth wrap at every orbit. This is the correct shape of the wrapped angle; if you want a monotone "cumulative" angle, derive the mean anomaly externally.

Overlay-safe. Yes.

9.1.4 Diff (pick 2 files)

These plots subtract one trajectory from another. Selecting one of them in the plot tree requires **exactly two** trajectory files selected in the file tree (sorted top-down = A then B); the dispatch applies the operation and renders the result.

If the two files share the same time grid sample-by-sample, the subtraction is direct. If they do not, B is automatically interpolated onto A's time grid restricted to the overlap window: position uses cubic Hermite interpolation with B's `v` as the analytical derivative, velocity uses linear per-component. The plot title gets a `(B interpolated)` suffix and the info label states the interpolation parameters so you know the diff is not direct.

Time-window mismatches with no overlap are reported with a clean message rather than a numpy traceback.

|Δr| (log y)

Formula. `|r_A(t) - r_B(t)|` on a logarithmic y axis.

When to use. The default error-magnitude plot for orbital regression. The log scale spans the typical many-orders-of- magnitude growth from sub-millimetre level (when B is an interpolation of a finer-grid reference) up to kilometre level over multi-day propagations.

|Δr| (linear y)

Same data on a linear axis. Better for inspecting the *shape* of the error growth near the end of the propagation, where the log scale flattens out.

|Δv| (log y)

Formula. `|v_A(t) - v_B(t)|`.

The velocity error sits typically two to three orders of magnitude below the position error for well-tuned propagators (the velocity field is smoother than the position field, and integrator errors accumulate quadratically in position vs linearly in velocity).

|Δv| (linear y)

Same data on a linear axis.

Δx, Δy, Δz per component

Per-component position difference: `A.x - B.x`, `A.y - B.y`, `A.z - B.z` on a shared axes with a legend.

When to use. When you want to see which coordinate axis dominates the error and at what frequency. Modulations periodic with the orbital period are common; secular drift on one component points at a specific perturbation mismatch.

RIC frame (radial/in-track/cross-track)

The most informative diff plot for orbital regression. The position-error vector is projected onto the RIC frame of trajectory A:

- **Radial** axis: `r_hat_A` (positive outward).
- **Cross-track** axis: `c_hat_A = (r_A × v_A) / |r_A × v_A|` (orbit normal, positive along `h`).
- **In-track** axis: `i_hat_A = c_hat × r_hat_A` (completes the right-handed frame; for circular orbits this aligns with the velocity direction).

The three components are plotted on a shared linear y axis with a legend.

Interpretation.

- A growing **in-track** component is the signature of a small **energy / timing drift** — the two propagators have slightly different mean motion and one drifts ahead of the other along the orbit. Typical for harmonics-degree mismatches.
- A growing **radial** component points at an **altitude error** — one orbit sits slightly higher than the other on average.
- A growing **cross-track** component points at an **out-of-plane / nodal drift** — difference in i or RAAN.

For a tuned high-fidelity setup on a near-circular orbit, the in-track component is the largest by an order of magnitude or more.

> *The "in-track" axis here is the standard RIC convention (Vallado's "S" axis, also called the Hill frame). It is **not** exactly the LVLH velocity axis, although for circular orbits the two coincide. See chapter 10, section 10.4 for the explicit relation.*

9.2 Accelerations plots (`SPDYACC_`)

The accelerations binary records the per-force accelerations at every record, alongside the total. These plots break the contributions down.

Total |a_total|

Formula. `|a_total| = sqrt(a_x^2 + a_y^2 + a_z^2)` on a logarithmic y axis.

When to use. Quickest sanity check on the order of magnitude of the total perturbing acceleration. For a low-lunar-orbit at N=80 harmonics, `|a_total|` is dominated by the central two-body term (a few `m/s^2` at the surface, dropping with altitude) and the harmonics contribution (smaller by an order of magnitude).

Overlay-safe. Yes.

Per-force breakdown (log y)

Each of the active force contributions (two-body, harmonics, each third body, SRP) is plotted as its own line on a logarithmic y axis with a legend.

When to use. The "who is doing what" plot — tells you which force dominates at every part of the orbit. Typical observations: SRP shows the eclipse cuts (zero during umbra, modulated by penumbra); harmonics oscillates with the spacecraft's orbital phase; third-body Earth dominates over third-body Sun for near-Moon orbits.

Overlay-safe. No (draws multiple lines per file).

Eclipse fraction

If `[events]` was enabled and the engine recorded the eclipse sunlight fraction at every step, this plot displays it on a linear y axis in `[0, 1]`. `1.0` = full sunlight, `0.0` = total umbra, intermediate values = penumbra.

Overlay-safe. Yes.

9.3 Events plots (`SPDYEVT_`)

The events binary records every detected event (IMPACT, ECLIPSE entry/exit) along the propagation.

Events timeline

A horizontal-axis-time scatter plot with one marker per detected event, colour-coded by event kind. The y axis is purely visual (separate row per kind for readability).

When to use. Quick visualisation of when, in the simulation window, each event happens. For impact-prediction work, the first IMPACT marker gives the predicted collision time at a glance.

Overlay-safe. No (multi-kind plot).

10. Frame conventions

A great many sign and orientation mistakes in orbital work come down to differing frame conventions. SpOdy uses a single small set of frames, all spelt out here.

10.1 The central-body inertial frame

The propagation frame is the **central body's inertial frame**, which the schema spells `central_inertial`. For a Moon-centred propagation (the only central body supported in this release):

- **Origin** at the centre of the Moon.
- **Axes** aligned with the ICRF (International Celestial Reference Frame), which is the modern realisation of the J2000 inertial axes. In practice the X axis points toward the dynamical equinox of J2000, the Z axis aligns with the J2000 mean rotation pole of Earth, and Y completes the right-handed triad.
- **Right-handed.**
- **No rotation** with respect to the distant background of galaxies; the Moon orbits, librates, and rotates *inside* this frame.

All TOML inputs (`initial_state.position_km`, `velocity_kms`) and all binary outputs are in this frame. The engine does not perform any frame conversions internally beyond the conversions needed to evaluate body-fixed gravity harmonics at each step.

> *If you have a state vector in a different frame (Earth-centred J2000, Moon's body-fixed frame, an ecliptic frame), convert it to the lunar-centred ICRF before pasting it into the TOML. The SPICE toolkit's `spkez` plus `pxform` calls are the canonical way to do this; the SpOdy bundle does not include a SPICE wrapper.*

10.2 The body-fixed frame

Internally, the engine evaluates the spherical-harmonic gravity expansion in the **Moon's principal-axis body-fixed frame**, which is the frame the GRGM1200B coefficient set is expressed in. The rotation from inertial to body-fixed is constructed at every step from the planetary ephemeris (rotation matrix from DE440's lunar attitude data). Users never see this frame: the inputs and outputs stay in the inertial frame, and the body-fixed conversion happens transparently inside the harmonics evaluation.

This is the relevant point for users: even though the engine uses the body-fixed frame in the harmonics math, you do **not** need to worry about libration angles, principal-axis vs mean-Earth axes, or sidereal-time alignment in your inputs. The conversion is exact for every step using the same

source data NASA uses for the GRGM1200B model.

10.3 The RIC frame (RIC = Radial / In-track / Cross-track)

This is the frame used by the **diff RIC** plot. It is built **on the fly at every sample** from the state vector of trajectory A:

Axis	Definition
Radial	`r_hat_A = r_A / |r_A|`, positive outward
Cross-track	`c_hat_A = (r_A × v_A) / |r_A × v_A|`, positive along the orbit normal
In-track	`i_hat_A = c_hat × r_hat_A`, completes the right-handed frame

The frame is right-handed; the axes are mutually orthogonal at every sample, but their inertial orientation rotates with the orbit. The frame is also called the **RSW frame** (Vallado's nomenclature) or the **Hill frame** (in the context of the Clohessy-Wiltshire linearised equations).

10.3.1 What goes where

When you decompose a position-error vector `Δr = r_A - r_B` onto this frame:

- A positive **radial** component means trajectory A is higher (farther from the central body) than B at that instant.
- A positive **in-track** component means A is *ahead* of B along the orbit. Persistent growth of `+` in-track is the canonical signal of a small energy / mean-motion drift between the two propagations.
- A positive **cross-track** component means A is offset out of B's instantaneous orbital plane along the orbit normal direction.

10.3.2 Comparison with LVLH

The Local Vertical / Local Horizontal frame (LVLH), used in spacecraft attitude work, is *related to but not identical with* the RIC frame. The signs differ:

	RIC (SpOdy)	LVLH
Z / W	`+r_hat`, outward	`-r_hat`, nadir (toward central body)
X / I	`c_hat × r_hat` (≈ `+v_hat` for circular orbits)	≈ `+v_hat`
Y / C	`+h_hat` (orbit normal)	`-h_hat`

If you need an LVLH decomposition you can compute it from the SpOdy diff RIC output by flipping the radial and cross-track signs. SpOdy does not provide an LVLH plot directly because RIC is the conventional choice in conjunction-assessment and orbit- regression work, and a sign-flipped duplicate plot would be just visual noise.

10.4 Classical orbital elements

The classical Keplerian elements are reported in the **central- body inertial frame**, with the central body's gravitational parameter `mu` baked in. For the Moon today, `mu = 4902.800066 km^3/s^2`. This number lives in the GUI source code as `MU_MOON_KM3_S2` and is the same value used in the C engine for the two-body reference acceleration.

10.4.1 Reference plane

Inclination, RAAN, and the argument of periapsis are reported relative to the **inertial XY plane** (the ICRF XY plane, projected at the Moon's centre). This is *not* the Moon's equatorial plane: the lunar pole is tilted ~5.1° from the ecliptic normal, and the ICRF Z axis aligns with Earth's mean pole, so the lunar equator is tilted ~24° from the ICRF XY plane.

A consequence: a near-polar lunar orbit (which is "polar" with respect to the Moon's body-fixed pole) appears at `i ≈ 85–115°` in SpOdy's elements, not at exactly 90°, because the reference plane is the ICRF and not the lunar equator.

This is the conventional choice and matches the SPICE convention when querying the LRO ephemeris in J2000. If you specifically need elements relative to the Moon's equator, post-process the inertial state vectors externally; SpOdy does not currently offer this view.

10.4.2 Quadrant resolution

The right quadrants are picked from the standard sign tests:

- **RAAN**: `Ω` is in `[0, π]` when the Y component of the node line is non-negative, in `(π, 2π)` otherwise.
- **Argument of periapsis**: `ω` is in `[0, π]` when the Z component of the eccentricity vector is non-negative, in `(π, 2π)` otherwise.
- **True anomaly**: `ν` is in `[0, π]` when `r · v ≥ 0` (i.e. moving away from periapsis), in `(π, 2π)` otherwise.

10.5 Sun direction (3D viewer)

The `+ Sun arrow` button in the 3D viewer computes the direction to the Sun **from the central body's centre** at the typed epoch, using a **low-precision analytic model** (Meeus-style series for the solar ecliptic longitude and the obliquity of the ecliptic, sufficient for visualisation accuracy of arc-minute level).

The arrow direction is rendered in the same `central_inertial` frame as the rest of the scene, so it lines up correctly with the orbit polylines. The arrow length is purely visual (scaled to the Moon sphere's radius); it does *not* represent astronomical distance.

For propagation purposes the engine uses the full-precision DE440 ephemeris — the analytic Sun direction is used only for the viewport arrow, never for any force computation.

11. Diff and validation workflow

The diff plots (chapter 9, section *Diff (pick 2 files)*) compare two trajectories sample by sample. This chapter explains how to use them in the two most common scenarios: regression between SpOdy runs, and validation against an external reference such as a SPICE ephemeris. It also documents the time-grid alignment rules and how to read the resulting numbers.

11.1 Two scenarios, same dispatcher

The same six diff plots apply to two distinct workflows:

1. **SpOdy vs SpOdy regression.** You ran a scenario twice with slightly different settings (different harmonics degree, with and without SRP, on different machines, or before and after a code change). The diff tells you how much the two runs disagree.
2. **SpOdy vs reference validation.** You have a reference trajectory in `SPDYOUT_` format produced by another tool — SPICE, a previously-validated engine, or any other source — and you want to know how close SpOdy gets.

The interaction is identical in both cases: load one file by clicking it (call this A), Ctrl-click the second (call this B) in the file tree to multi-select, then click a diff leaf in the plot tree.

11.2 A is the loaded file, B is the other

The two-file selection has a small but consequential rule: the **currently-loaded file is treated as A**, and the other selected file is **B**. Concretely:

- The title and info label show `A = <basename>`, `B = <basename>` so you can always read off which is which.
- The RIC frame is built from A's state. Out-of-plane errors are measured against A's instantaneous orbital plane, not B's.
- Time-grid alignment (next section) is done **B onto A's grid**, so the rendered samples are at A's times.

Swap which is which by clicking the other file in the tree (making it the loaded one); the previous loaded file remains in the selection so the second Ctrl-click is not needed.

11.3 Time-grid alignment

The diff dispatcher checks the two arrays' time columns before subtracting. Two paths:

11.3.1 Fast path: aligned grids

If A and B share the same time grid sample-by-sample (every pair of times matches within 1 ms), both arrays pass through unchanged and the diff is computed directly. This is the case when both runs were produced by SpOdy at the same `output.mode = fixed` with the same `output.interval_s`, or are otherwise known to be on the same fixed grid.

The plot title shows `A = … B = …` without further annotation.

11.3.2 Slow path: cubic-Hermite interpolation

If the grids do not match, B is interpolated onto A's grid restricted to the **overlapping time window**:

- **Position** uses cubic Hermite interpolation, with B's velocity vector as the analytical derivative at each sample. This gives C^1 continuity at sample points and is exact for any polynomial position trajectory up to cubic order.
- **Velocity** uses per-component linear interpolation.

The plot title appends `(B interpolated)` and the info label records the parameters in the form `B interpolated onto A's grid (46053 -> 8640 samples, cubic Hermite on r, linear on v)`.

When the two time windows have no overlap at all (one run started after the other ended), the dispatcher reports `no overlap` cleanly and refuses to render anything.

11.3.3 Why cubic Hermite

For trajectory data sampled at coarse intervals relative to the orbital period, linear interpolation introduces a curvature error of order `(v &Delta;t)^2 / R`. For LRO at 11 s sampling that is about 170 m of interpolation error per sample — much bigger than the m-level diff signal we want to measure.

Cubic Hermite interpolation with `v` as the analytical derivative catches the position curvature exactly to cubic order, reducing the interpolation error to negligible levels for any practical sampling cadence. We use linear interpolation for velocity because velocity is smoother than position (one fewer derivative of the underlying acceleration field), and linear interpolation of v is adequate at the precision the diff plot needs.

11.4 Reading the magnitudes

Concrete numbers for the example scenario shipped with SpOdy:

11.4.1 LRO 6-day, N=80 harmonics, vs SPICE LRO reference

Quantity	Value
Duration	6 days
Sample count	~8000 (spody) vs ~46000 (SPICE)
`|Δr|` at t=0	0 m exact (shared IC)
`|Δr|` at t=6d	~110 m
`|Δr|` max	~1.2 km at mid-window
`|Δr|` mean	~250 m
`|Δv|` max	~0.8 mm/s

The growth pattern of `|Δr|` is **not monotone**: it accumulates, peaks somewhere in the middle of the propagation, and recovers toward the end. This is characteristic of the gravity-harmonics mismatch between SpOdy's degree-80 expansion and SPICE's reconstructed ephemeris, which uses the full GRGM1200B set.

11.4.2 RIC interpretation for the same diff

On the same example, the RIC decomposition shows the in-track component growing roughly linearly to ~1 km mid-window, while the radial and cross-track components stay below 200 m. This is the canonical signature of a small **mean-motion drift** induced by truncating the harmonics expansion at degree 80 — the two propagations have slightly different effective orbital period because they capture slightly different fractions of the non-spherical potential.

Raising `harmonics_degree` to 150 reduces the in-track drift proportionally; raising it beyond 200 yields diminishing returns because the GRGM1200B coefficients become weakly observed at high degrees and adding terms can slightly increase the residual.

11.5 Combining diff plots through the tile dashboard

A useful pattern for the validation workflow is to tile the four diff plots into a 2×2 dashboard for a single overview shot:

1. Click A (your run) in the file tree to load it.
2. Ctrl-click B (your reference) so both are selected.
3. In the plot tree, Ctrl-click each of `|Δr| (log y)`, `|Δv| (log y)`, `Δx, Δy, Δz per component`, and `RIC frame`.
4. The `▦ Tile selected (4)` button at the bottom of the plot tree lights up. Click it.

The canvas splits into a 2×2 grid showing all four diffs at once. The shared subtitle reports A, B, and the `(B interpolated)` annotation if applicable.

11.6 Validating against an external reference

The recommended workflow when you have an external reference trajectory in `SPDYOUT_` format:

1. Place the reference file in the same folder as your TOML (or in any folder of your choice).
2. Open the TOML in the Run tab and run the propagation.
3. Switch to the Analysis tab. The working directory is already pointed at your TOML's parent.
4. Click your reference file in the file tree to load it.
5. Ctrl-click the SpOdy output file you just produced.
6. Click `RIC frame` in the plot tree.

If your reference file is not yet in `SPDYOUT_` format (e.g. you have it as a CSV of (t, r, v) rows, or as a SPICE BSP kernel), you need to convert it first — SpOdy's reader path accepts only the binary format. The format is documented in chapter 12, section *Output binary formats*; producing a converter for it is typically a 30-line numpy script.

11.7 Validating against another SpOdy run

The simplest regression scenario: run the same TOML twice and diff. The two runs will be byte-identical unless something has changed between them (the engine is deterministic for a given input). A non-zero diff therefore unambiguously points at the change between the two runs — a different harmonics degree, a recompiled engine, a different machine.

This is also the cheapest way to check that a code change you just made does not silently affect the physics: run before, run after, diff. The diff plot showing flat zero across all four panels is the convincing assertion that nothing changed.

12. Command-line reference

The engine `spody.exe` is a standalone command-line tool. The GUI drives it through subprocess calls, but you can drive it yourself — from a PowerShell prompt, from a Python script, from a CI pipeline. This chapter documents the subcommands and their flags.

The conventions throughout the chapter:

- All path arguments are resolved relative to the **current working directory** of the shell that launches `spody.exe`, *except* for the paths inside a TOML, which are resolved relative to the **TOML's directory**.
- The engine returns **exit code 0** on success and **non-zero** on any failure (parse error, validation error, runtime error). A human-readable error message is printed to stderr before exit.

12.1 Global structure

```
spody.exe <command> [arguments]
```

The five commands are:

Command	Purpose
`validate`	parse + sanity-check an input TOML, no run
`propagate`	run a single-case simulation
`batch`	run a multi-case sweep
`convert`	data-file conversions (ephemeris today)
`info`	print version + build info

Pass `--help` or no command to print a short usage summary.

12.2 `spody validate`

```
spody.exe validate <input.toml>
```

Loads the TOML, runs the engine's parser and the validator, and prints either:

- a multi-line summary of the parsed configuration starting with `OK`, on success;
- a single-line `error: <file>:<reason>` message on failure.

The validator runs the same checks the **Validate** button in the GUI triggers, so the verdicts agree by construction. Useful in scripts to gate downstream work on a clean input.

Exit codes.

- `0` — the TOML is well-formed and internally consistent.
- `1` — parse error, validation error, or unreadable file.

12.3 `spody propagate`

```
spody.exe propagate <input.toml> [--out <dir>]
```

Reads a TOML describing a single scenario, integrates the trajectory, and writes the output files specified in the `[output]` block.

Flags:

- `--out <dir>` (optional) — override the `output.*_file` paths' parent directory. Useful when you want to dispatch the same TOML against many output folders without editing the file.

Stdout/stderr. The engine prints a header line and per-step diagnostics to stdout, with errors on stderr. The GUI streams this into the terminal pane; on the command line you see it in your shell.

Exit codes.

- `0` — the propagation reached `duration_s` without an unrecoverable error.
- `1` — parse error, validation error, integrator failure (step size driven below `h_min_s`), or I/O error.

12.4 `spody batch`

```
spody.exe batch <input.toml> [--out <dir>]
```

Reads a TOML with a `[batch]` block and runs the parameter sweep described by `batch.cases_file`. The TOML must contain a `[batch]` section; running `propagate` against a batch TOML is an error, and vice versa.

Flags:

- `--out <dir>` (optional) — override `batch.output_dir`.

Threading. Reads `batch.thread_number` and dispatches that many concurrent cases through OpenMP. Use `1` for sequential execution; the engine handles any value up to the host's logical- CPU count.

Exit codes.

- `0` — every case completed successfully.

- `1` — one or more cases failed; the engine prints a summary line `batch stopped at case N/M after T s` and exits.

12.5 `spody convert`

The umbrella command for data-file conversions. Today only the ephemeris conversion is implemented.

12.5.1 `spody convert ephemeris`

```
spody.exe convert ephemeris <folder> <de_family> <date1> [date2 ...]
```

Converts the JPL ASCII DE-family chunks present in `<folder>` into the internal `.spody` binary format SpOdy uses for ephemeris queries.

Arguments:

- `<folder>` — directory containing the `header.<de_family>` file and the `ascpXXXXX.<de_family>` chunks. The output `de<de_family>.spody` is written into the **same folder**.
- `<de_family>` — the DE-family identifier (`440` for DE440, the only supported value today).
- `<date1> [date2 ...]` — one or more chunk identifiers (without the `ascp` prefix and the `.<de_family>` suffix). Order does not matter; the converter sorts them internally.

This is the same conversion the setup wizard runs automatically on first launch (chapter 3, section 3.5). You typically invoke it manually only when you have downloaded additional chunks outside the wizard or want to regenerate `de440.spody` for any other reason.

Example.

```
spody.exe convert ephemeris .\data\DE440 440 01950 02050
```

Reads `data\DE440\header.440`, `data\DE440\ascp01950.440`, and `data\DE440\ascp02050.440`. Writes `data\DE440\de440.spody`.

Exit codes.

- `0` — conversion succeeded.
- `1` — missing file, unreadable folder, or library failure.

12.6 `spody info`

```
spody.exe info
```

Prints the SpOdy application version, the engine library version, the engine library's git commit hash, and the build timestamp.

No flags; no error path.

12.7 Examples

Quickly validating every example TOML at once on PowerShell:

```
Get-ChildItem .\examples -Recurse -Filter input.toml | ForEach-Object {
    .\spody.exe validate $_.FullName
}
```

Re-running the LRO 6-day example in a fresh output folder:

```
.\spody.exe propagate .\examples\lro_6day\input.toml --out C:\Temp\lro_run
```

Dispatching a batch across all logical CPUs from a shell:

```
# Set thread_number = N inside the TOML, then:
.\spody.exe batch .\examples\batch_demo\input.toml
```

12.8 Output binary formats

For completeness, the three binary formats SpOdy writes are documented here so external tools (Python notebooks, CI pipelines) can read them without going through the GUI.

Every binary starts with the same **24-byte header**:

Offset	Bytes	Content
0	8	ASCII magic (no NUL terminator)
8	4	uint32 little-endian version
12	4	uint32 little-endian payload metadata
16	8	reserved (two zeroed uint32)

The three magics are `SPDYOUT_` for trajectories, `SPDYACC_` for the per-force accelerations breakdown, and `SPDYEVT_` for events. The payload metadata is interpreted per format:

12.8.1 `SPDYOUT_` — trajectory

- Payload: `state_dim` (always `6` today, meaning `r, v`).
- Record: 7 little-endian doubles in order `t, x, y, z, vx, vy, vz`.

- Record size: 56 bytes.

12.8.2 `SPDYACC_` — per-force accelerations

- Payload: record size in bytes (varies with the number of active forces).
- Record: a header double `t`, followed by per-force triples `ax, ay, az` for each active force in the order documented in the engine's source. The total `a_total` triple and the `eclipse_fraction` scalar are included.

12.8.3 `SPDYEVT_` — events

- Payload: record size in bytes (80).
- Record: an `EVENT_KIND` uint32, a flags uint32, the event time as a double, and a 64-byte body whose layout depends on the kind (IMPACT, ECLIPSE_ENTER, ECLIPSE_EXIT).

A NumPy-based Python reader for these formats is the standard companion to the GUI and lives in the `spody_io` package distributed alongside the GUI bundle. Importing it from a script gives you `read_trajectory`, `read_accelerations`, and `read_events` functions that return structured `ndarray`s.

13. Troubleshooting

This chapter is a catalogue of the issues you are most likely to hit, grouped by the part of the workflow they belong to, with the recommended fix for each. If you encounter something not listed here please file a report with the steps that reproduce it; the catalogue grows with experience.

13.1 Setup wizard

13.1.1 "Download failed" on a DE440 chunk

The JPL FTP-over-HTTPS server occasionally returns transient errors during high-traffic periods. The standard fix is to:

1. Wait a minute.
2. Click **Download** on the affected card again.

If the failure is persistent, check the URL printed in the error dialog against the canonical pattern `https://ssd.jpl.nasa.gov/ftp/eph/planets/ascii/de440/ascpXXXXX.440` and adjust it in the card's URL field if the server has moved.

13.1.2 "Download failed" on a GRGM1200B file

The PDS Geosciences Node at Washington University hosts the GRGM1200B coefficient files. If the canonical URL (`https://pds-geosciences.wustl.edu/grail/grail-l-lgrs-5-rdr-v1/grail_1001/shadr/`) returns a 404, the file has been relocated. Open the PDS Geosciences GRAIL landing page in a browser, navigate to the `shadr/` subfolder, find the current location of `gggrx_1200b_sha.tab` and `gggrx_1200b_sha.lbl`, and paste the new URL into the card.

13.1.3 "Conversion failed (exit 1)"

The automatic conversion runs `spody.exe convert ephemeris` under the hood. The two common failures:

- **Missing files**: at least one ASCII chunk was not actually downloaded successfully. Hit **Refresh** at the bottom of the wizard and verify every required card is green. Re-download any that are not.
- `spody.exe` **is not configured**: the wizard delegates the conversion to the engine, and if the engine path is empty or invalid the conversion cannot run. Open **Settings › Paths**, set the path to `spody.exe` (it lives next to `spody-gui.exe` in the bundle), and reopen the wizard.

13.1.4 Wizard reopens at every launch

This means the **hard run-guard** decides one or more required files are not in place. Common causes:

- The data folder is on a removable drive that is not currently mounted.
- A virus scanner quarantined the downloaded files (some enterprise scanners are aggressive about large binary downloads from FTP-derived URLs).
- The bundle was moved to a different folder after the wizard was completed, and the registered data folder no longer points at a writable location.

Open the wizard and inspect the status of every card to identify the missing file; restore the file or change the data folder via the **Change…** button.

13.2 TOML form

13.2.1 "Form has invalid values" placeholder in the preview

A field's value cannot be coerced to its declared type — typically a non-numeric character in a float or integer field. Look for fields with the red border and fix the offending value. The TOML preview restores automatically once the form is again in a valid state.

13.2.2 **Validate** badge shows `(spody binary not set)`

The engine path is not configured. Open **Settings › Paths** and point at `spody.exe`.

13.2.3 **Validate** badge shows `(data not ready)`

The hard run-guard intercepted the Validate call because one or more required data files are missing from the data folder. The button next to the badge invites you to open the wizard; do so and complete the downloads.

13.2.4 **Validate** badge shows `(form has invalid values)`

The form contains a value that cannot be serialised. This is distinct from "out of range": the type conversion itself failed (e.g. typing letters in a numeric field). Fix the indicated field; the badge clears on the next edit.

13.2.5 **Validate** badge stays red after a fix

The previous red badge persists until you press **Validate** again. There is no auto-revalidation on edit; only a successful explicit Validate turns the badge green.

13.2.6 Floats render in scientific notation after a load

This is intentional. The form reads the float, then re-emits it through Python's `repr()` to ensure a round-trippable representation; the result may look like `1e-5` even if you typed `0.00001`. The two are equal. No information is lost.

> *Earlier versions of the form attached a `QDoubleValidator` that aggressively normalised user input (`1e-5` ⇒ `1e-05`, `0.00001` ⇒ `1e-05`). The current version intentionally does not validate floats with a Qt validator and keeps the typed text verbatim until the next Load.*

13.3 Run mode

13.3.1 "spody binary not set"

The engine path is empty. Open **Settings › Paths**.

13.3.2 "Cannot run: data not ready"

Same as the validate counterpart: the run-guard intercepted the launch because data files are missing. The dialog offers to open the wizard; accept the offer and complete the downloads.

13.3.3 Run starts but stops within seconds

Open the terminal pane on the right; the engine printed a one-line diagnostic before exit. Common cases:

- `error: harmonics_file: ENOENT` — the `[force_model].harmonics_file` path inside the TOML does not resolve. Remember the path is relative to the TOML's directory.
- `error: ephemeris.file: ENOENT` — same for the ephemeris path.
- `error: integrator step h dropped below h_min_s` — the adaptive controller could not maintain `rel_tol` and gave up. Try a smaller `rel_tol` (so the controller is happier with larger steps) or a smaller `h_min_s`.

13.3.4 Run is much slower than expected

Two common amplifiers:

- A **harmonics degree** above 200 in a long propagation; cost scales as O(N²), so going from N=80 to N=200 makes the engine 6× slower without proportional accuracy gain.
- A **tight** `rel_tol` (smaller than 1e-13) below the engine's resolution; the controller spends all its time rejecting steps.

Tune both first; engine-side performance is rarely the actual bottleneck.

13.3.5 Terminal pane stops updating mid-run

A long-running step (heavy harmonics evaluation, especially with event detection enabled) can keep the engine inside one numeric routine without producing console output for several seconds. The status bar still increments the elapsed-time counter, which is your hint that the process is alive even

when the terminal is quiet. Worry only if the elapsed-time counter freezes too.

13.4 Analysis tab

13.4.1 File tree shows no files in the working directory

The recursive `.bin` scan goes three folders deep. A binary buried deeper does not appear; either move the file up or use the **+ Add external file…** button to register it explicitly.

13.4.2 "Unknown file" on a `.bin` you know is good

The kind detection reads the first 8 bytes of the file and matches them against the four supported magics (`SPDYOUT_`, `SPDYACC_`, `SPDYEVT_`). A `.bin` produced by an external tool that does not write a SpOdy magic will be flagged; convert the file into one of the supported formats first.

13.4.3 Plot button is missing

There is no plot button. Click a leaf in the plot tree to render it; the dispatch is on the click itself.

13.4.4 Sun-arrow row is missing

The Sun-arrow row appears only when the **active plot is 3D**. The currently-selected leaf in the plot tree is 2D (which covers every plot except `3D orbit + Moon`); pick the 3D plot and the row reappears.

13.4.5 "Diff needs exactly 2 files (currently selected: 1)"

You clicked a diff leaf with only the loaded file in the selection. **Ctrl-click** another file in the file tree to add it to the selection, then re-click the diff leaf.

13.4.6 "Diff requires overlapping time windows"

The two selected files were propagated over disjoint time spans. This is fundamental: there is no way to compare two trajectories that do not coexist in time. Re-run one of them so the time windows overlap.

13.4.7 "Tile cannot mix single-file and diff plots"

The tile dispatcher requires the selection to be uniform: either all single-file specs (rendered against the loaded file) or all diff specs (rendered against the two-file selection). Clear the selection (Ctrl-click to unselect) and rebuild it.

13.4.8 3D viewer is unresponsive

The first VTK render after a fresh launch can take a few seconds while OpenGL is initialised. Subsequent renders are fast. If the viewer remains unresponsive after ten seconds, your graphics adapter may have an unusual OpenGL driver; switch to a 2D plot to confirm the rest of the application is healthy.

13.4.9 Moon sphere is flat grey instead of textured

The 3D scene falls back to the flat-grey sphere when no Moon texture is configured. Open **Settings › Paths › Moon texture (3D view)** and point at an equirectangular Moon image (JPEG or PNG). The texture is reloaded on the next plot dispatch; no application restart is needed.

13.5 Settings

13.5.1 Persistent paths are not remembered across launches

Settings are stored in the Windows registry under `HKEY_CURRENT_USER\Software\SpOdy\SpOdy`. If you are running as a guest or restricted user, registry writes may be blocked by policy. Run the application as your normal interactive user.

13.5.2 Resetting the application to a clean state

Two steps:

1. Delete the `data/` folder next to `spody-gui.exe`.
2. Delete the registry key `HKEY_CURRENT_USER\Software\SpOdy\SpOdy` via `regedit`.

The next launch behaves exactly as a fresh install: the wizard pops, no recent files are remembered, no paths are configured.

14. Glossary

A short list of the technical terms used through the manual, defined where SpOdy uses them. Cross-references to the chapter that introduces each term in detail are given in square brackets.

Accelerations binary. — A `SPDYACC_`-format output file listing, at every record, the contribution of each active force to the total acceleration. Used by the **Per-force breakdown** plot. [Chapter 9.]

Accelerations file. — The `output.accelerations_file` TOML field, naming the on-disk accelerations binary. [Chapter 6.]

Adaptive step. — The mode of the RKDP integrator in which the step size `h` is chosen at every step to maintain `rel_tol`. Opposed to fixed-step integration. [Chapter 6.]

A/m. — Area-to-mass ratio, in m²/kg. The single parameter SpOdy needs to compute SRP acceleration when used in debris mode; in spacecraft mode it is derived from `area_m2 / mass_kg`. [Chapter 6.]

Argument of periapsis (`ω`). — The classical orbital element measuring the angle from the ascending node to the periapsis, in the orbital plane. [Chapter 9.]

Batch. — A multi-case parameter sweep, driven by a CSV file and a column-to-target mapping. [Chapter 7.]

Cases file. — The CSV file describing one case per row, referenced from `batch.cases_file`. [Chapter 7.]

Central body. — The body the propagation is centred on (today only the Moon is supported). Defines the inertial frame the state vector is expressed in. [Chapter 6, chapter 10.]

Central-inertial frame. — The reference frame the engine propagates in: origin at the central body's centre, axes aligned with the ICRF (J2000), right-handed. [Chapter 10.]

Coverage profile. — The user's choice of ephemeris time window in the setup wizard: *Modern era* (1950 – 2050) or *Full pack* (1550 – 2650). [Chapter 3.]

`Cr`. — Coefficient of reflectivity. `1.0` = pure absorbing, `2.0` = pure mirror, intermediate values for partial diffuse reflection. SpOdy uses the cannonball SRP model with a single Cr. [Chapter 6.]

Cross-track. — The third RIC axis: perpendicular to the orbit plane, positive along the angular-momentum direction. [Chapter 10.]

Cubic Hermite. — Interpolation scheme used by the diff dispatcher when the two trajectories are on different time grids. Uses position and velocity at each sample to produce a piecewise cubic that matches both at sample points. [Chapter 11.]

Data dir. — The folder the wizard populates with the external data files. By default a `data/` subfolder next to `spody-gui.exe`, overridable via the wizard. [Chapter 3.]

DE440. — The JPL planetary ephemeris model SpOdy uses for third-body positions. Distributed as ASCII chunks; the wizard converts these into the internal `de440.spody` binary. [Chapter 3, chapter 6.]

Delta mode. — The batch column mapping mode in which the CSV cell value is *added* to the TOML's nominal value, as opposed to *overriding* it. Useful for Monte-Carlo perturbations. [Chapter 7.]

Diff plot. — A plot in the `Diff (pick 2 files)` folder of the plot tree. Subtracts trajectory B from trajectory A sample-by-sample (with cubic Hermite interpolation when the grids do not match). [Chapter 9, chapter 11.]

Eccentricity (`e`). — The classical orbital element measuring how non-circular the orbit is. `e = 0` for circular, `0 < e < 1` for elliptical, `e = 1` for parabolic, `e > 1` for hyperbolic. [Chapter 9.]

Eclipse threshold. — The sunlight-fraction value at which an eclipse event fires. Configured in `[events]`. [Chapter 6.]

Engine. — The C executable `spody.exe` that runs the actual numerical integration. Distinct from the GUI (`spody-gui.exe`), which is a Python application that drives the engine. [Chapter 1.]

Ephemeris. — Time-tabulated planetary positions and velocities used to compute third-body gravity. SpOdy uses JPL's DE440 in an internal binary format. [Chapter 3, chapter 6.]

Event. — A discrete occurrence detected by the engine during a propagation: an IMPACT against a celestial body surface, or (when enabled) an ECLIPSE entry/exit. [Chapter 6.]

`et_start_s`. — The start epoch of the simulation, in TDB seconds past J2000. [Chapter 6.]

Form. — The Run tab's left pane: one widget per TOML field, with range checks and live preview. [Chapter 5.]

`GRGM1200B`. — The recommended lunar spherical-harmonic gravity coefficient set, derived from the NASA GRAIL mission's observations. SpOdy reads the PDS-distributed `.tab` file at runtime. [Chapter 3.]

Hard run-guard. — The runtime check that refuses to launch the engine if any required data file is missing. It also blocks the **Validate** button for the same reason. [Chapter 3, chapter 13.]

Harmonics degree. — The truncation order `N` of the spherical-harmonic gravity expansion. Higher = more accurate but O(N²) more expensive. [Chapter 6.]

ICRF. — International Celestial Reference Frame, the modern realisation of the J2000 inertial axes. SpOdy's central-inertial frame is ICRF-aligned. [Chapter 10.]

In-track. — The second RIC axis: perpendicular to the radial direction, in the orbital plane, in the half-plane containing the velocity. For circular orbits this aligns with the velocity direction. [Chapter 10.]

Inclination (`i`). — The classical orbital element measuring the orbit plane's tilt with respect to the reference plane (the ICRF XY plane in SpOdy). [Chapter 9.]

Inline table. — The TOML syntax `{ key = value, &hellip; }`, used in `[batch.columns]` for delta-mode column descriptors. [Chapter 7.]

J2000. — The standard epoch 2000-01-01 12:00:00 TT, used as the zero of TDB-seconds time scale. [Chapter 6.]

`mu`**.** — Gravitational parameter, in km³/s². For the Moon: `4902.800066`. [Chapter 9.]

Override mode. — The batch column mapping mode in which the CSV cell value *replaces* the TOML's nominal value. The default. [Chapter 7.]

Overlay-safe. — Property of a plot that draws exactly one line per file, so an N-file overlay produces N lines (legible) rather than 3N or more (illegible). [Chapter 9.]

Plot tree. — The lower-half tree in the Analysis tab's left column, listing the plots applicable to the currently-loaded file's kind. [Chapter 8.]

RAAN (`Ω`). — Right Ascension of the Ascending Node, the classical orbital element measuring the angle from the reference X axis to the orbit's ascending node. [Chapter 9.]

Radial. — The first RIC axis: from the central body to the spacecraft, positive outward. [Chapter 10.]

`rel_tol`**.** — The relative error tolerance the integrator maintains per accepted step. Smaller = more accurate, slower. [Chapter 6.]

RIC. — Radial / In-track / Cross-track. A local orbit frame defined sample-by-sample from the state vector. Used by the RIC diff plot. Equivalent to the RSW frame in Vallado's nomenclature and the Hill frame in Clohessy-Wiltshire studies. [Chapter 10.]

RKDP / RKDP45. — Runge-Kutta Dormand-Prince 5(4), the adaptive integrator SpOdy uses. The 5/4 numbers refer to the order of the embedded error estimate. [Chapter 6.]

Run-guard. — See *Hard run-guard*.

SPDYOUT_ / SPDYACC_ / SPDYEVT_. — The 8-byte magics of SpOdy's three binary output formats. [Chapter 12.]

SPICE. — NASA NAIF's toolkit and associated kernels. SpOdy's planetary ephemeris is derived from SPICE-format DE440 data; the recommended validation reference for any SpOdy run is a SPICE-derived trajectory of the same epochs. [Chapter 11.]

SRP. — Solar radiation pressure. SpOdy uses the cannonball model: a single A/m and Cr, with eclipse cuts when enabled. [Chapter 6.]

Step mode. — The `output.mode = "step"` value: one output record per accepted RKDP step, irregular sampling. Opposed to fixed-step output. [Chapter 6.]

Target. — A dotted path inside the TOML schema that a batch column can override. E.g. `spacecraft.mass_kg`, `debris.am_srp`, `initial_state.position_km[0]`. [Chapter 7.]

TDB. — Barycentric Dynamical Time, the time scale the DE440 ephemeris is expressed in. SpOdy's `et_start_s` is in TDB seconds past J2000. [Chapter 6.]

Third body. — A celestial body whose gravity perturbs the central-body two-body solution but which is not itself propagated. Configured in `force_model.third_bodies`. [Chapter 6.]

Tile dashboard. — The multi-subplot rendering mode triggered by the ▦ **Tile selected (N)** button: N plots from the multi-selection are drawn as a grid in a single matplotlib figure. [Chapter 8.]

TOML. — The input file format SpOdy reads. A human-readable, strongly-typed configuration syntax; see https://toml.io/ for the specification. [Chapter 6.]

True anomaly (`v`). — The classical orbital element measuring the angle from the periapsis to the current position, in the orbital plane, measured in the direction of motion. [Chapter 9.]

Two-body. — The Kepler-problem central acceleration `-mu * r / |r|^3`. Always present; the rest of the force model is added on top. [Chapter 6.]

Validate. — The action of running `spody.exe validate` against a TOML to check it parses and passes consistency rules, without actually propagating. [Chapter 5, chapter 12.]

Vis-viva. — The relation `|v|^2 = mu * (2/|r| - 1/a)` that ties together speed, distance, and semi-major axis on a Keplerian orbit. Used by the orbital-elements solver to derive `a` from the state vector. [Chapter 9.]

Working directory. — The folder the Analysis tab scans for `.bin` files. Set explicitly via **Change…** or auto- filled from the loaded TOML's parent. [Chapter 8.]